

CLASES ABSTRACTAS.

Piensen en una clase abstracta como un plano incompleto de una casa. El plano define la estructura básica (por ejemplo, que hay habitaciones, un baño y una cocina), pero no todos los detalles (como el color de las paredes o el tipo de grifo). Alguien más, un constructor, debe tomar ese plano y completarlo para hacer una casa habitable.

Concepto: ¿Qué es una Clase Abstracta?

Una **clase abstracta** es una clase que *no puede ser instanciada* directamente. Su propósito es servir como una base para otras clases, definiendo una estructura y comportamiento comun, pero dejando algunos métodos sin implementar. Estos métodos se conocen como **métodos abstractos**, y son los que las clases "hijas" deben completar.

Una clase abstracta puede contener:

- **Métodos abstractos:** Métodos sin implementación, solo con la firma (`public abstract void hacerAlgo();`).
- **Métodos concretos:** Métodos con implementación completa, como los de una clase normal.
- **Constructores y atributos:** Puede tener campos y constructores, aunque no puede ser instanciada directamente.

La principal diferencia con una interfaz es que una clase abstracta puede tener métodos con un cuerpo implementado y atributos, mientras que una interfaz no. Sin embargo, ambas promueven la reutilización de código y la creación de un contrato de comportamiento.

Aplicación: ¿Cuándo usar una Clase Abstracta?

Las clases abstractas son ideales cuando tienes un conjunto de clases estrechamente relacionadas que comparten una base común. Por ejemplo:

- **Jerarquías de objetos:** Si tienes una jerarquía de clases como `Animal` -> `Perro`, `Gato`, `Pajaro`, la clase `Animal` podría ser abstracta, ya que un "animal" genérico no existe, pero sí sus subcategorías. `Animal` podría tener un método `comer()` (con implementación) y un método abstracto `hacerSonido()` (ya que cada animal hace un sonido diferente).
 - **Plantillas de diseño:** Si varios objetos siguen una plantilla de pasos, una clase abstracta puede definir el esqueleto del algoritmo con métodos concretos, y dejar que las subclasses implementen los pasos específicos que varían.
-

Ejemplo: La Granja de Animales

```
public abstract class Animal {  
    private String nombre;  
    private int edad;  
  
    public Animal(String nombre, int edad) {  
        this.nombre = nombre;  
        this.edad = edad;  
    }  
    //Metodo concreto  
    public void comer() {  
        System.out.println(this.nombre + " está comiendo.");  
    }  
  
    // Método abstracto que las subclasses deben implementar  
    public abstract void hacerSonido();  
}
```

Creamos clases concretas que extienden `Animal`

```
public class Vaca extends Animal {  
    public Vaca(String nombre, int edad) {  
        super(nombre, edad);  
    }  
}
```

```

    @Override
    public void hacerSonido() {
        System.out.println("Muuu");
    }
}

public class Cerdo extends Animal {
    public Cerdo(String nombre, int edad) {
        super(nombre, edad);
    }

    @Override
    public void hacerSonido() {
        System.out.println("Oink oink");
    }
}

```

Ahora, ¿cómo lo usamos?

```

public class Granja {
    public static void main(String[] args) {
        // No podemos crear una instancia de Animal directamente
        // Animal animalGenerico = new Animal("...", 0); // Esto daría
un error de compilación

        // Pero sí podemos usar sus subclases
        Vaca miVaca = new Vaca("Lola", 3);
        Cerdo miCerdo = new Cerdo("Porky", 1);

        miVaca.comer();
        miVaca.hacerSonido();

        System.out.println();

        miCerdo.comer();
    }
}

```

```
        miCerdo.hacerSonido();  
    }  
}
```

Salida:

```
Lola está comiendo.  
Muuu  
  
Porky está comiendo.  
Oink oink
```

LAS INTERFACES.

Piensen en una interfaz como un contrato o un manual de instrucciones que una clase se compromete a seguir. Si una clase dice "implemento esta interfaz", está prometiendo que tendrá los métodos y propiedades definidos en ese contrato.

Concepto: ¿Qué es una Interfaz?

Una **interfaz** es un tipo de dato abstracto que define un conjunto de métodos que una clase debe implementar. A diferencia de las clases, una interfaz no puede ser instanciada directamente y no tiene variables de instancia (atributos) con valores. En Java, una interfaz solo puede contener:

- **Métodos abstractos:** Son los métodos "sin cuerpo" que las clases que implementan la interfaz deben definir.
- **Constantes:** Campos public static final.
- **Métodos default y static:** A partir de Java 8, se pueden añadir métodos con implementación para no romper la compatibilidad con las clases que ya la usaban.

Las interfaces promueven un concepto clave: **la programación a la interfaz, no a la implementación**. Esto significa que nos preocupamos por "qué" hace un objeto (sus métodos) y no por "cómo" lo hace (su implementación interna).

Aplicación: ¿Cuándo usar una Interfaz?

Las interfaces son perfectas para definir roles o capacidades que diferentes clases pueden tener, sin importar su jerarquía de herencia. Algunas de las aplicaciones más comunes incluyen:

- **Definir un comportamiento común:** Si varias clases (por ejemplo, Perro, Gato, Pajaro) necesitan hacer algo similar (como hacerSonido()), una interfaz PuedeHacerSonido es ideal.
- **Programar a la interfaz:** Cuando trabajas con colecciones de objetos heterogéneos (ej., una lista de Vehiculos donde cada uno puede ser un Coche, una Moto o un Avion), puedes usar una interfaz PuedeArrancar para asegurar que todos tengan el método arrancar().
- **Implementar múltiples herencias de comportamiento:** Java no permite la herencia múltiple de clases, pero sí permite que una clase implemente múltiples interfaces. Esto te permite "heredar" comportamientos de varias "fuentes".

Ejemplo: El Mundo de la Animación

Imaginemos que estamos desarrollando un juego y necesitamos que diferentes personajes (un Heroe, un Monstruo) puedan "atacar" a otros. Definimos una interfaz Atacable.

```
public interface Atacable {  
    void recibirDanio(int cantidad);  
    boolean estaVivo();  
}  
  
public class Heroe implements Atacable {  
    private int vida;
```

```

public Heroe(int vida) {
    this.vida = vida;
}

@Override
public void recibirDanio(int cantidad) {
    this.vida -= cantidad;
    System.out.println("Héroe ha recibido " + cantidad + " de daño.
Vida restante: " + this.vida);
}

@Override
public boolean estaVivo() {
    return vida > 0;
}
}

```

Ahora a implementar esa Interface

```

public class Monstruo implements Atacable {
    private int vida;

    public Monstruo(int vida) {
        this.vida = vida;
    }

    @Override
    public void recibirDanio(int cantidad) {
        this.vida -= cantidad;
        System.out.println("Monstruo ha recibido " + cantidad + " de
daño. Vida restante: " + this.vida);
    }

    @Override
    public boolean estaVivo() {
        return vida > 0;
    }
}

```

Ahora podemos usar estas clases de forma genérica, sin importar si son un Heroe o un Monstruo.

```
public class Juego {  
    public static void main(String[] args) {  
        Atacable heroe = new Heroe(100);  
        Atacable monstruo = new Monstruo(50);  
  
        System.out.println(";Comienza la batalla!");  
        monstruo.recibirDanio(20);  
        heroe.recibirDanio(30);  
  
        System.out.println("\nEstado de la batalla:");  
        System.out.println("Héroe está vivo? " + heroe.estaVivo());  
        System.out.println("Monstruo está vivo? " +  
monstruo.estaVivo());  
    }  
}
```
